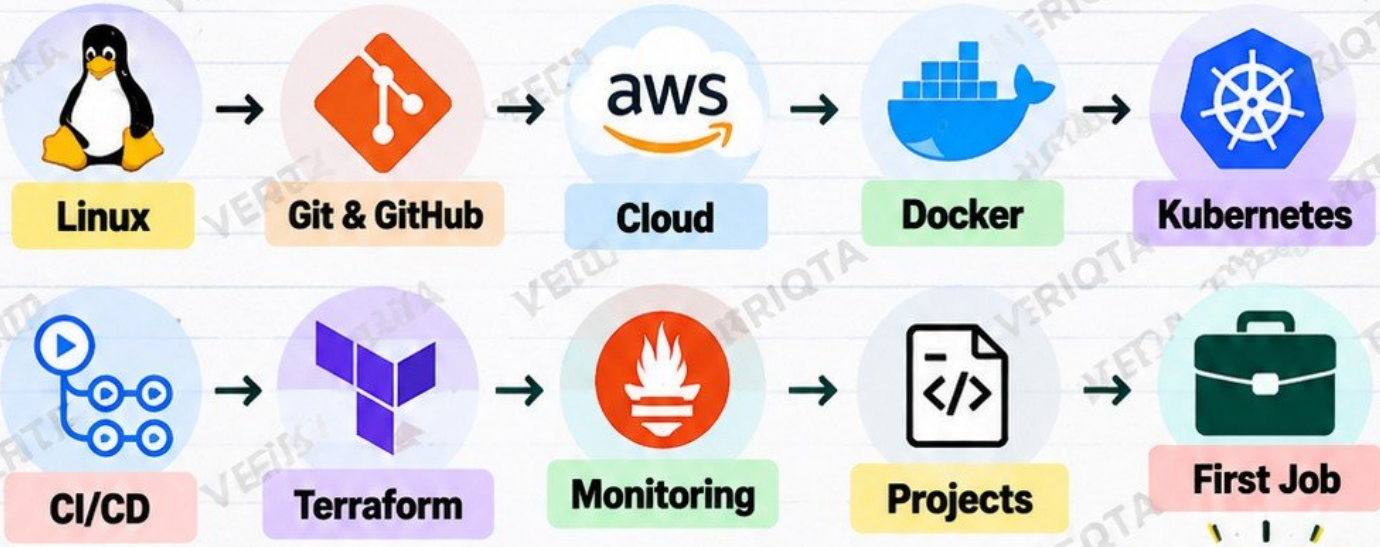
VERIQTA

DevOps Roadmap: From Zero to First Job

A Step-by-Step Guide to Learn, Practice and Build Real Skills for Your First DevOps Role in 2026





@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

What Is DevOps and What Does a DevOps Engineer Do?

1. What Is DevOps?

- DevOps is a combination of **people, processes** and **automation.**
- It brings **development (Dev)** and **operations (Ops)** teams together.
- The goal is to build, test and deliver software faster, more reliably and safely.
- DevOps is not a tool. It is a way of working and a set of practices.

2. Why Does It Exist?

- Traditional teams often worked in silos.
- Developers write code and hand it over to operations.
- This causes slow releases, errors and miscommunication.
- DevOps exists to improve collaboration, automation and speed while maintaining stability and security.

3. The Software Delivery Lifecycle

Continuous Feedback

Code	Build	Test	Deploy	Monitor
Developers write and update code.	Code is compiled or packaged into an artifact.	Automated tests check for errors and quality.	The application is released to staging or production.	The application is monitored for performance, errors and availability.

4. How Dev and Ops Work Together

Development Team (Dev)

- Write and update code
- Add new features
- Fix bugs
- Work with version control
- Collaborate on releases

Collaboration
Shared Goals
Automation
Faster Delivery
Better Quality

Operations Team (Ops)

- Manage infrastructure
- Keep systems reliable
- Monitor applications
- Handle security and access
- Support deployments
- Respond to incidents

5. What Does a Junior DevOps Engineer Do?

- Set up and maintain development and deployment environments.
- Write and improve automation scripts.
- Work with CI/CD pipelines.
- Help deploy applications.
- Monitor systems and investigate issues.
- Work with cloud platforms and containers.
- Collaborate with developers and operations teams.
- Learn and improve existing infrastructure.

6. Key DevOps Practices

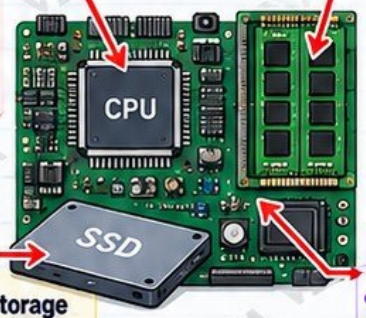
- Version control (e.g. Git)
- Continuous integration and continuous delivery (CI/CD)
- Infrastructure as code (e.g. Terraform)
- Monitoring and logging
- Collaboration and communication
- Automation and repeatability

7. Remember This

- DevOps is people + process + automation.
- Dev and Ops work together toward the same goal.
- The lifecycle goes from code to monitoring with continuous feedback.
- A DevOps engineer helps build, deploy, monitor and improve systems.
- The goal is faster, safer and more reliable software.

How Computers, Servers, and Operating Systems Work

1. The Main Components of a Computer



CPU
- Executes instructions
- Performs calculations

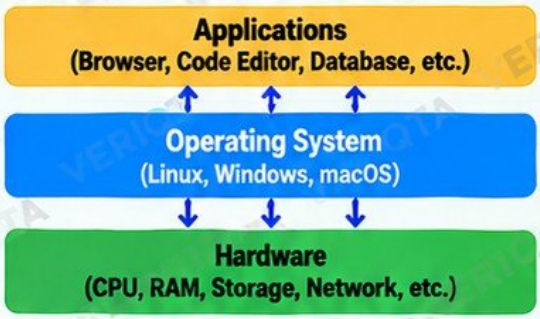
RAM
- Temporary memory
- Stores data for running apps

Storage
- Long-term storage
- Stores OS, apps and files
- SSD (faster)
- HDD (larger, slower)

Motherboard
- Connects all components
- Allows them to communicate

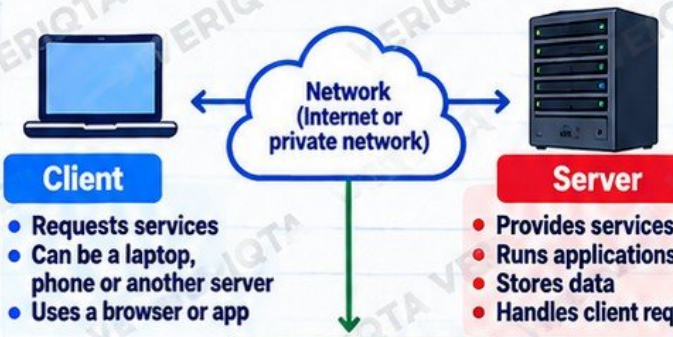
2. What Does an Operating System Do?

An operating system (OS) manages the computer's hardware and provides services for applications.



- Manages CPU, memory, storage and devices.
- Runs and isolates applications.
- Provides file systems, user managementt and security.
- Handles networking and communication.
- Controls access to hardware resources.
- Keeps the system stable and efficient.

3. Servers, Clients, and Applications



Client
- Requests services
- Can be a laptop, phone or another server
- Uses a browser or app

Server
- Provides services
- Runs applications
- Stores data
- Handles client requests

Application
- Software that performs a specific task
- Runs on a server or a client
- Examples: web app, database, file server

4. Types of Servers

Physical Server



- Actual hardware in a data center
- Dedicated resources
- Expensive to buy and maintain
- Used for high performance needs

Virtual Machine (VM)



- Runs on a physical server using a hypervisor
- Has its own OS
- Shares hardware resources
- More flexible and cheaper than physical

Cloud Server



- Runs in the cloud (e.g. AWS, Azure)
- On-demand resources
- Scalable and pay-as-you-use
- No physical hardware to manage

5. Key Concepts to Remember

- **CPU:** Executes instructions and performs calculations.
- **RAM:** Temporary memory for running applications.
- **Storage:** Long-term storage for OS, apps and files.
- **Operating System:** Manages hardware and runs applications.
- **Server:** Provides services to clients.
- **Client:** Requests services from a server.
- **Application:** Software that performs a specific task.

6. Practice: Inspect Your System Information

Command	What It Shows
`uname -a`	OS name, version and kernel
`hostname`	Your server or computer name
`cat /etc/os-release`	Detailed OS information (Linux)
`lscpu`	CPU details
`free -h`	Memory (RAM) usage
`lsblk`	Storage devices and partitions
`df -h`	Disk space usage
`ps aux | head`	Running processes (first few)

Linux Fundamentals Every DevOps Beginner Needs

1. What Is Linux?

- Linux is a free and open-source operating system.
- It manages hardware, runs applications and provides tools for DevOps work.
- Most servers, cloud environments and containers use Linux.

2. Linux Filesystem Structure



The main directory. All files and folders start from here.

- Essential commands
- Configuration files
- User home directories
- User applications
- Logs and variable data
- Temporary files
- Optional software

3. Absolute vs Relative Paths

Absolute Path

- Starts from the root directory (/)
- Always points to the same location
- Example: /home/user/projects

Relative Path

- Starts from your current directory
- Changes based on where you are
- Example: projects/file.txt

4. Common Linux Commands

Command	What It Does	Example
pwd	Shows current directory	pwd
ls	Lists files and directories	ls -l
cd	Changes directory	cd /home/user
mkdir	Creates a new directory	mkdir devops
cp	Copies files or directories	cp file.txt backup.txt
mv	Moves or renames files	mv old.txt new.txt
cat	Displays file content	cat file.txt

5. Practical Examples

1. Check your current location

```
$ pwd
```

Shows the full path of your current directory.

2. List files with details

```
$ ls -l
```

Lists all files and directories with permissions and size.

3. Create and navigate to a directory

```
$ mkdir devops
$ cd devops
```

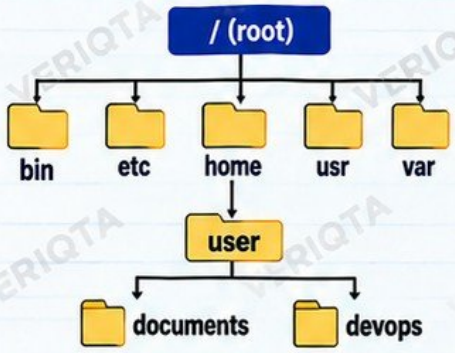
Creates a folder named devops and moves into it.

4. Create a file and view its content

```
$ echo "Hello DevOps" > file.txt
$ cat file.txt
```

Creates a file with text and displays the content.

6. Navigating the Filesystem (Example)



- Use cd to move between directories.
- Use .. to go up one level.
- Use . to stay in the current directory.
- Use / to go to root.

7. Common Errors and Troubleshooting

Error Message	What It Means	How to Fix It
No such file or directory	The file or path does not exist.	Check the path and spelling using ls.
Permission denied	You do not have permission.	Use the correct user or sudo if needed.
Is a directory	You tried to open a directory as a file.	Use ls to view it instead of cat.
Command not found	The command is not installed.	Check the spelling or install the package.

8. Practice: Try These Commands

1. Run pwd and note your current directory.
2. Use ls to list the files in your home directory.
3. Create a folder named practice and navigate into it.
4. Create a file named notes.txt and add some text.
5. Copy the file to backup.txt and then rename it to final.txt.
6. Move back to your home directory and delete the practice folder.

9. Key Takeaways

- Linux uses a directory structure with / as the root.
- You can navigate, create, copy, move and view files using simple commands.
- Absolute paths always start from /.
- Relative paths depend on your current location.
- Most file and directory errors are caused by incorrect paths, permissions or typos.

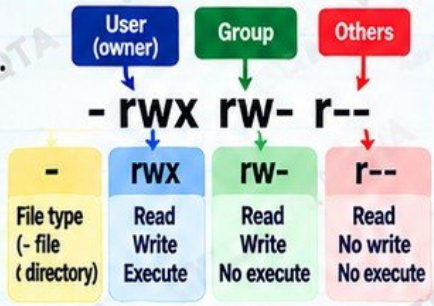
@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

VERIQTA

Linux Permissions, Processes, and Services

1. Users, Groups and File Permissions

- **Users:** Individual accounts that can log in and run commands.
- **Groups:** Collections of users that share permissions.
- **Permissions:** Control who can read, write or execute a file or directory.



2. Common Permission Commands

Command	What It Does	Example
chmod	Changes file permissions	chmod 755 script.sh
chown	Changes file owner	chown user file.txt
chgrp	Changes file group	chgrp devops file.txt
ls -l	Shows permissions and ownership	ls -l
umask	Sets default permissions for new files	umask 022

3. Understanding chmod Permissions

Permissions can be set using numbers or symbols.

r = 4	w = 2	x = 1
Read View file or list directory	Write Modify file or create files	Execute Run a file or enter a directory

Number	Permission	Meaning
755	rwx r-x r-x	Owner full access, others read and execute
644	rw- r-- r--	Owner read/write, others read only
700	rwx --- ---	Owner only
775	rwx rwx r-x	Owner and group full, others read and execute

4. Processes: ps and top

Command	What It Does
ps aux	Lists all running processes
ps -ef	Shows processes in full format
top	Real-time process monitor (press q to quit)
htop	Interactive process monitor (if installed)
kill <PID>	Stops a process (use with care)

5. Services and systemctl

- **Services** are long-running applications managed by the operating system.
- **systemctl** is used to start, stop, enable and check the status of services.
- Common **services:** web servers, databases, container runtimes, monitoring agents and more.

Command	What It Does	Example
systemctl status	Checks service status	systemctl status nginx
systemctl start	Starts a service	systemctl start nginx
systemctl stop	Stops a service	systemctl stop nginx
systemctl restart	Restarts a service	systemctl restart nginx
systemctl enable	Starts on boot	systemctl enable nginx
systemctl disable	Stops on boot	systemctl disable nginx

6. Why Least Privilege Matters

- Give users only the permissions they need.
- Reduces the risk of accidental or malicious changes.
- Protects system files and critical servicses.
- Helps maintain system stability and security.
- Use **sudo** only when necessary.
- Avoid running daily work as root.

7. Common Errors and Fixes

Error Message	What It Means	How to Fix It
Permission denied	You do not have the required permission.	Use correct permissions or sudo if needed.
No such file or directory	The file or path does not exist.	Check the path and spelling.
Failed to start service	The service could not be started.	Check logs: journalctl -xe and fix the error.
Process not found	The process is not running or wrong PID.	Verify with ps aux and use the correct PID.

8. Practice: Try These Commands

1. Create a new user and add it to a group.
2. Create a file, change its permissions and owner.
3. Use ps aux and top to find running processes.
4. Check the status of a service (e.g. nginx or ssh).
5. Stop, start and enable a service.

9. Key Takeaways

- Linux uses users, groups and permissions to control access.
- Use chmod, chown, ps, top and systemctl for daily work.
- Services run in the background and can be managed with systemctl.
- Least privilege improves security and stability.
- Read error messages carefully. They help you find the real problem.

Networking Fundamentals for DevOps

1. What Is a Network?

- A network connects computers so they can communicate and share data.
- Devices use IP addresses to find each other.
- Protocols like TCP, UDP, HTTP, HTTPS and SSH define how data is sent.
- DNS converts domain names (e.g. google.com) to IP addresses.

2. How a Web Request Works (High Level)



1. You enter example.com in your browser
2. DNS returns the server's IP address
3. Your computer connects to the server using a protocol (e.g. HTTPS)
4. The server sends the website data back to your browser

3. IP Addresses

- An IP address is a unique number that identifies a device on a network.
- IPv4 example: 192.168.1.10
- IPv6 example: 2001:db8::1

Type	Example	Used For
Localhost	127.0.0.1	Your own computer
Private IP	192.168.1.10	Internal networks (not accessible from internet)
Public IP	8.8.8.8	Accessible from the internet

4. Common Protocols

Protocol	Port(s)	Purpose
TCP	–	Reliable, connection-oriented protocol
UDP	–	Faster, no connection (used for real-time traffic)
HTTP	80	Unencrypted web traffic
HTTPS	443	Encrypted web traffic
SSH	22	Secure remote access to servers

5. Common Ports

Port	Service
22	SSH
80	HTTP
443	HTTPS
3306	MySQL
5432	PostgreSQL
6379	Redis
8080	Alternative HTTP
8443	Alternative HTTPS

6. Useful Networking Commands

Command	What It Does	Example
ping	Tests basic connectivity	`ping 8.8.8.8`
curl	Sends an HTTP request	`curl -I https://example.com`
ss	Shows open ports and connections	`ss -tuln`
dig	Queries DNS records	`dig example.com`
nslookup	Looks up DNS information	`nslookup example.com`
traceroute	Shows the path to a host	`traceroute example.com`

7. Example: Checking a Website

```
# 1. Check connectivity
$ ping example.com
# 2. Check the IP address
$ dig example.com
# 3. Check the HTTP response
$ curl -I https://example.com
# 4. Check open ports on your server
$ ss -tuln
# 5. Trace the network path
$ traceroute example.com
```

8. Common Network Errors and Fixes

Error Message	What It Means	How to Fix It
Connection refused	The service is not running on that port.	Check if the service is running and the port is correct.
Connection timed out	The server did not respond.	Check network, firewall, security groups or routing.
Name or service not known	DNS cannot find the domain.	Check DNS with dig or nslookup.
No route to host	Network cannot reach the destination.	Check IP address, routing and firewall rules.
Permission denied	You do not have permission (e.g. SSH or file).	Use the correct user, key or permissions.

9. Key Takeaways

- Devices communicate using IP addresses.
- DNS converts domain names to IP addresses.
- TCP and UDP are common network protocols.
- Different services use different ports.
- Use tools like ping, curl, ss and dig to test and troubleshoot connectivity.
- Network issues are often caused by DNS, firewalls, wrong ports or services not running.

10. Practice

1. Ping a website: ping google.com
2. Find its IP address: dig google.com
3. Check the HTTP response: curl -I https://google.com
4. List open ports: ss -tuln
5. Try accessing a closed port and observe the error.

Git and GitHub for Real Engineering Work

1. What Is Version Control?

- Version control tracks changes to your code over time.
- It allows multiple people to work on the same project.
- You can go back to previous versions if needed.
- **Git** is a distributed version control system.
- **GitHub** is a cloud platform to host Git repositories and collaborate with others.

2. How Git and GitHub Work Together

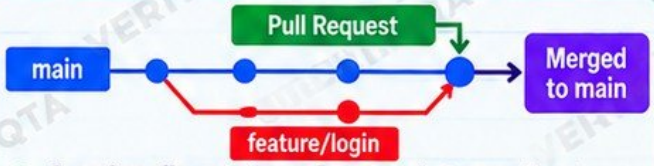


- You work locally using **Git**.
- You push your changes to **GitHub**.
- Others can pull, review and contribute.

3. Essential Git Commands

Command	What It Does	Example
`git init`	Initialize a new repository	`git init`
`git clone`	Copy a remote repository	`git clone <url>`
`git add`	Stage changes	`git add file.txt`
`git commit`	Save changes with a message	`git commit -m "message"`
`git push`	Upload changes to GitHub	`git push origin main`
`git pull`	Download latest changes	`git pull origin main`
`git status`	Check the status of changes	`git status`
`git log`	View commit history	`git log --oneline`

4. Branches, Pull Requests, and Merge Conflicts



- Branches allow you to work on new features without affecting the main code.
- A **pull request (PR)** lets you propose your changes for review.
- Team members review the code, suggest changes and approve it.
- **Merge conflicts** happen when two people change the same part of a file. You must resolve the conflict before merging.

5. Why Code Reviews Are Important

- Catch bugs and security issues early.
- Improve code quality and consistency.
- Share knowledge across the team.
- Ensure the code follows best practices.
- Build safer and more reliable software.



6. Practice: Create a Project and Push to GitHub

```
# 1. Create a project folder and initialize Git
mkdir devops-project
cd devops-project
git init
# 2. Create a file and add content
echo "Hello DevOps" > README.md
# 3. Stage and commit the file
git add README.md
git commit -m "Initial commit"
# 4. Create a repository on GitHub (empty)
# Then push your code
git branch -M main
git remote add origin https://github.com/yourusername/devops-project.git
git push -u origin main
```

7. Common Merge Conflict Example

File from main branch

```
1  name = "DevOps"
2  version = "1.0"
3  environment = "prod"
```

File from feature branch

```
1  name = "DevOps"
2  version = "2.0"
3  environment = "staging"
```

Merged File (Conflict)

```
1  name = "DevOps"
2  <<<<<<< main
3  version = "1.0"
4  =======
5  version = "2.0"
6  >>>>>>> feature
7  environment = "staging"
```

Steps to Fix

1. Open the conflicted file.
2. Compare the changes.
3. Keep the correct content.
4. Remove the conflict markers.
5. Save and commit the file.

8. Practice Tasks

1. Create a new GitHub repository.
2. Initialize a local project and add files.
3. Make at least 3 commits with meaningful messages.
4. Create a new branch, make changes and open a pull request.
5. Review a pull request (on your own or with a friend).
6. Resolve a merge conflict.
7. Push the final code to the main branch.

9. Key Takeaways

- Git tracks changes and allows collaboration.
- GitHub hosts your repositories and enables teamwork.
- Branches and pull requests help manage changes safely.
- Code reviews improve quality and prevent problems.
- These are essential skills for real engineering work.

Bash Scripting and Automation Basics

1. What Is a Shell Script?

- A shell script is a text file with a series of commands.
- It helps you automate repetitive tasks.
- Engineers use scripts to save time, reduce errors and standardize work.
- Scripts are run by a shell (e.g. Bash).
- You can use scripts for system administration, deployments, backups and monitoring.

2. Script Structure and Running a Script

```
#!/bin/bash
# This is a comment
echo "Hello DevOps"
echo "This is my first script"
```

- **Shebang** (tells the system to use Bash)
- **Comment** (ignored by the shell)
- **Commands** (executed by the shell)

How to run a script:

1. Save the file (e.g. script.sh)
2. Make it executable: `chmod +x script.sh`
3. Run it: `./script.sh`

3. Variables and Input

Example	Description	Output
`name="DevOps"`	Create a variable	Stores text
`echo $name`	Print the variable	DevOps
`read -p "Enter your name: " user`	Take user input	Stores input in user
`$((a + b))`	Simple arithmetic	Result of calculation

4. Conditions (if Statement)

```
if [ $1 -gt 10 ]; then
  echo "Number is greater than 10"
else
  echo "Number is 10 or less"
fi
```

- Use if to run commands based on a condition.
- Common operators:
  - `-gt` greater than
  - `-lt` less than
  - `-eq` equal to
  - `-ne` not equal to

5. Loops (for and while)

For loop (loop through a list)

```
for i in 1 2 3; do
  echo "Iteration $i"
done
```

While loop (repeat until condition)

```
count=1
while [ $count -le 5 ]; do
  echo "Count: $count"
  count=$((count + 1))
done
```

6. Exit Codes

```
echo "Running a command..."
ping -c 1 google.com
if [ $? -eq 0 ]; then
  echo "Ping successful"
else
  echo "Ping failed"
fi
```

- Every command returns an exit code.
- 0 means success (✅).
- Non-zero means failure (❌).
- You can use exit codes to check if a command worked and take action.

7. Practical Example: System Health Script

```
#!/bin/bash

echo "=== System Health Check ==="
echo "Date: $(date)"

echo "Uptime:"
uptime

echo -e "\nDisk Usage:"
df -h

echo -e "\nMemory Usage:"
free -h

echo -e "\nTop 5 Processes by CPU:"
ps aux --sort=-%cpu | head -n 6
```

What This Script Does:

- Shows the current date and time.
- Displays system uptime.
- Shows disk usage.
- Shows memory usage.
- Lists top 5 processes using the most CPU.

8. Make the Script Executable and Run It

Command	What It Does
`chmod +x system-check.sh`	Makes the script executable
`ls -l system-check.sh`	Shows the permissions
`./system-check.sh`	Runs the script

9. Practice Task: Automate a Repetitive Task

1. Create a script that creates a folder, navigates into it and creates 3 files.
2. Create a script that checks if a website is reachable (use `ping` or `curl`).
3. Create a script that cleans old log files (e.g. delete files older than 7 days).
4. Add conditions and error handling using exit codes.
5. Make your script executable and run it.

10. Key Takeaways

- ✅ Bash scripts help you automate repetitive tasks.
- ✅ You can use variables, conditions, loops and exit codes.
- ✅ Scripts must have executable permissions to run.
- ✅ You can use scripts for system administration, deployments and monitoring.
- ✅ Practice writing small scripts and improve them over time.

Cloud Computing and AWS Fundamentals

1. What Is Cloud Computing?

- Cloud computing provides computing resources over the internet.
- You can use servers, storage, databases and more without owning physical hardware.
- You pay only for what you use.
- It allows you to scale quickly and deploy applications globally.
- Major cloud providers include AWS, Microsoft Azure and Google Cloud.

2. How Cloud Computing Works



User (Your Computer) Accesses a service via the internet

Internet Sends the request to the cloud

Cloud Server (EC2, etc.) Processes the request and runs your application

Application (Website, API, etc.) Returns the response to the user

User → Internet → Cloud Server → Application

You access a cloud server over the internet, the server runs your application, and the response is sent back to you.

3. Shared Responsibility Model

AWS Manages (Cloud Provider)	You Manage (Your Responsibility)
Physical servers	Operating system
Data centers	Applications
Networking infrastructure	Security configurations
Hardware	Your data
Virtualization	Access and permissions (IAM)
Availability of cloud services	Network rules (e.g. security groups)

4. Regions and Availability Zones



- A Region is a geographic location (e.g. us-east-1).
- Each region has multiple Availability Zones (AZs).
- AZs are separate data centers with independent power, networking and cooling.
- Using multiple AZs improves availability and fault tolerance.

5. Key AWS Services for DevOps

Amazon EC2

- Virtual servers in the cloud.
- Run Linux or Windows instances.
- Used for web servers, applications and more.

Amazon S3

- Object storage for files.
- Store and retrieve any amount of data.
- Used for backups, static websites, images and more.

Amazon VPC

- A virtual private network in the cloud.
- Control your network (subnets, route tables, security groups).
- Keep your resources isolated and secure.

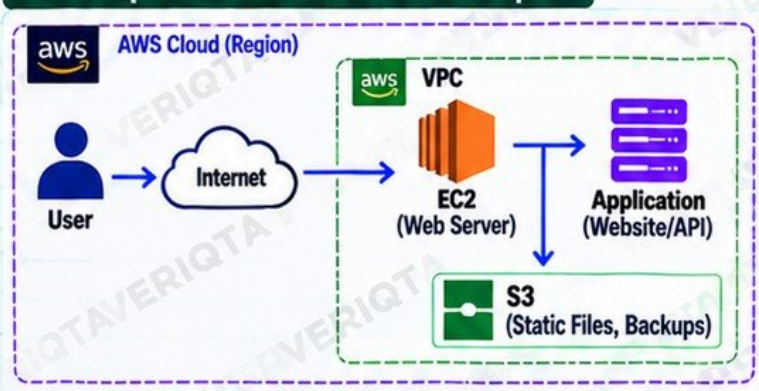
AWS IAM

- Manage users, groups and permissions.
- Control who can access which AWS resources.
- Use least privilege for security.

6. Core Cloud Concepts

Concept	What It Means
Compute	Virtual servers (e.g. EC2) that run your applications.
Storage	Store data (e.g. S3 for files).
Networking	Control network access (e.g. VPC, subnets, security groups).
Identity	Manage users and permissions (e.g. IAM).
Scalability	Increase or decrease resources as needed.
High Availability	Use multiple AZs to stay online even if one fails.

7. Simple AWS Architecture Example



8. Practice: Explore AWS with the Free Tier

1. Create an AWS account.
2. Launch a free-tier EC2 instance (Amazon Linux).
3. Connect to it using SSH.
4. Create an S3 bucket and upload a file.
5. Explore the VPC and security group settings.
6. Clean up your resources to avoid unexpected charges.

9. Key Takeaways

- Cloud computing provides on-demand computing resources.
- AWS uses regions and availability zones for global infrastructure.
- EC2, S3, VPC and IAM are core services for DevOps work.
- You are responsible for security in the cloud (shared responsibility).
- Understanding compute, storage, networking and identity is essential for deploying and managing applications.

Build and Secure Your First Cloud Server

1. What You Will Do

- Understand SSH and key-based authentication.
- Learn basic cloud firewall and security-group rules.
- Connect to a Linux virtual machine.
- Install and run a simple web server.
- Verify connectivity and investigate common SSH failures.
- Understand cloud costs and resource cleanup.

2. High-Level Architecture



Use SSH with your private key	Traffic allowed only on specific ports	Security group allows SSH (22) and HTTP (80)	Access the website using the server's public IP

3. Create a Cloud Server (AWS EC2 Example)

1. Sign in to AWS and launch an EC2 instance.
2. Choose an Amazon Linux or Ubuntu image.
3. Select an instance type (e.g. t2.micro for learning).
4. Create or use an existing key pair (downloads a .pem file).
5. Configure the security group (allow SSH and HTTP).
6. Launch the instance.



4. Security Group Rules (Firewall)

Type	Protocol	Port Range	Source	Purpose
SSH	TCP	22	Your IP (or 0.0.0.0/0 for learning)	Allow SSH access
HTTP	TCP	80	0.0.0.0/0	Allow web traffic
HTTPS	TCP	443	0.0.0.0/0	Allow secure web traffic

5. SSH and Key-Based Authentication

```
# Set correct permissions on your key file
chmod 400 my-key.pem

# Connect to your server
ssh -i my-key.pem ec2-user@YOUR_PUBLIC_IP

# For Ubuntu, the user is usually 'ubuntu'
ssh -i my-key.pem ubuntu@YOUR_PUBLIC_IP
```

- Key-based authentication is more secure than passwords.
- Keep your private key file safe and never share it.
- Use chmod 400 to restrict permissions on your key file.

6. Install and Run a Web Server (Nginx Example)

```
# Update packages
sudo apt update
# Install Nginx
sudo apt install -y nginx
# Start and enable Nginx
sudo systemctl start nginx
sudo systemctl enable nginx
# Check status
sudo systemctl status nginx
```

Verify It Works

1. Open a browser.
2. Go to: http://YOUR_PUBLIC_IP
3. You should see the Nginx welcome page.

NGINX

7. Verify Connectivity

```
# Test network connectivity
ping YOUR_PUBLIC_IP

# Check open ports
curl -I http://YOUR_PUBLIC_IP

# Check SSH connection
ssh -v -i my-key.pem ec2-user@YOUR_PUBLIC_IP
```

- ping checks basic connectivity.
- curl tests if the web server is responding.
- ssh -v shows detailed connection logs.

8. Common SSH Failures and Fixes

Error Message	What It Means	How to Fix It
Permission denied (publickey)	Wrong key, or key permissions are open.	Use the correct key and chmod 400 your-key.pem.
Connection timed out	Port 22 is blocked.	Check security group rules and your network.
No route to host	Wrong IP or instance is stopped.	Verify the public IP and instance status.
Host key verification failed	The server key changed.	Remove old key from ~/.ssh/known_hosts.

9. Cloud Costs and Resource Cleanup

- Stop or terminate instances when not in use.
- Remove unused storage (e.g. EBS volumes).
- Delete unnecessary snapshots.
- Monitor your AWS bill and set up billing alerts.
- Use free tier eligible services for learning.



10. Key Takeaways

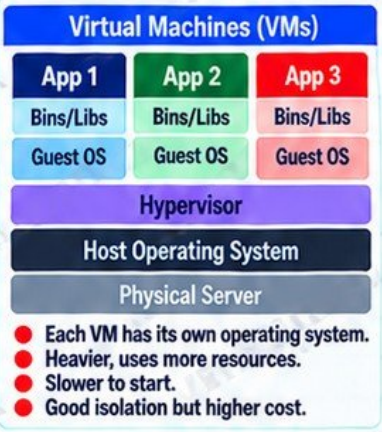
- You can launch and connect to a cloud server using SSH and a key pair.
- Security groups act as a firewall to control incoming traffic.
- You can install and run a web server to host a simple website.
- Always verify connectivity and troubleshoot errors using logs and tools.
- Be mindful of cloud costs and clean up your resources.

Docker Fundamentals: Images and Containers

1. Why Do We Use Containers?

- Containers package an application and its dependencies together.
- They run consistently in any environment (development, testing, production).
- They are lightweight, fast and portable.
- They solve the "it works on my machine" problem.
- They make it easier to deploy, scale and manage applications.

2. Containers vs Virtual Machines



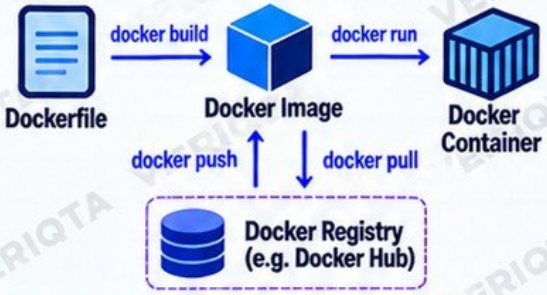
- Each VM has its own operating system.
- Heavier, uses more resources.
- Slower to start.
- Good isolation but higher cost.

- Share the host operating system.
- Lightweight and fast.
- Start in seconds.
- Use fewer resources.
- Easier to deploy and scale.

3. Key Docker Concepts

Image	Container	Dockerfile	Registry
A read-only template that contains an application and its dependencies.	A running instance of an image.	A text file with instructions to build an image.	A place to store and share images (e.g. Docker Hub).

4. Docker Architecture



5. Common Docker Commands

Command	What It Does	Example
docker build	Builds an image from a Dockerfile	`docker build -t myapp .`
docker run	Creates and starts a container	`docker run -d -p 80:80 nginx`
docker ps	Lists running containers	`docker ps`
docker ps -a	Lists all containers	`docker ps -a`
docker logs	Shows container logs	`docker logs <container_id>`
docker stop	Stops a running container	`docker stop <container_id>`
docker rm	Removes a container	`docker rm <container_id>`

6. Example: Build and Run a Simple Web App

```
# 1. Create a project folder
mkdir docker-demo && cd docker-demo
# 2. Create a simple index.html
echo "<h1>Hello from Docker!</h1>" > index.html
# 3. Create a Dockerfile
    FROM nginx:alpine
    COPY index.html /usr/share/nginx/html
    EXPOSE 80
# 4. Build the image
docker build -t my-nginx .
# 5. Run the container
docker run -d -p 8080:80 my-nginx
# 6. Verify
Open your browser: http://localhost:8080
```

7. Image Layers



Layer 4: Application files (e.g. index.html)
Layer 3: Configuration (e.g. nginx setup)
Layer 2: Dependencies (e.g. system libraries)
Layer 1: Base image (e.g. nginx:alpine)

- Images are made of multiple read-only layers.
- Each layer represents a set of changes.
- Layers are cached to make builds faster.
- When you rebuild, only changed layers are updated.

8. Container Lifecycle



9. Practice Tasks

1. Create a simple Nginx container and access it in your browser.
2. Build a custom image using a Dockerfile.
3. Run a container in detached mode (-d).
4. View container logs.
5. Stop and remove the container.
6. Push your image to Docker Hub (optional).

10. Key Takeaways

- Containers package applications and dependencies.
- They are lightweight, portable and fast.
- An image is a template; a container is a running instance.
- Dockerfiles define how to build images.
- Registries store and share images.
- Understanding image layers and the container lifecycle is essential for real-world DevOps work.

Docker Networking, Volumes, and Troubleshooting

1. What You Will Learn

- Understand container ports and port publishing.
- Learn persistent storage with Docker volumes.
- Understand container networking.
- Diagnose containers that exit unexpectedly.
- Practice running and inspecting a containerized web application.

2. Container Ports and Port Publishing



Run a container with port publishing:

```
docker run -d -p 8080:80 --name web nginx
```

- 8080 is the host port.
- 80 is the container port.
- You can access the app in your browser using http://localhost:8080

3. Docker Volumes (Persistent Storage)

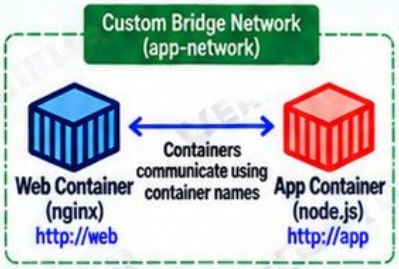


- Volumes store data outside the container.
- Data remains even if the container stops or is deleted.
- Useful for databases, logs, and persistent files.

Create and use a volume:

```
docker volume create my-data
docker run -d -v my-data:/usr/share/nginx/html \
  --name web nginx
```

4. Docker Networking



Create a network and run containers:

```
docker network create app-network
docker run -d --name web \
--network app-network nginx
docker run -d --name app \
--network app-network node:18
```

- Containers on the same network can talk to each other by name.
- Bridge network is the default and commonly used.
- You can use custom networks to isolate applications.

5. Example: Run a Containerized Web Application

```
# 1. Create a volume for website files
docker volume create web-data

# 2. Run Nginx container with port publishing and volume
docker run -d --name my-web -p 8080:80 \
  -v web-data:/usr/share/nginx/html nginx

# 3. Verify it is running
docker ps

# 4. Access it in your browser
http://localhost:8080

# 5. View logs
docker logs my-web
```

What This Does:

- Creates persistent storage for website files.
- Runs an Nginx container.
- Publishes container port 80 to host port 8080.
- You can access the website in your browser.
- Logs help you troubleshoot issues.

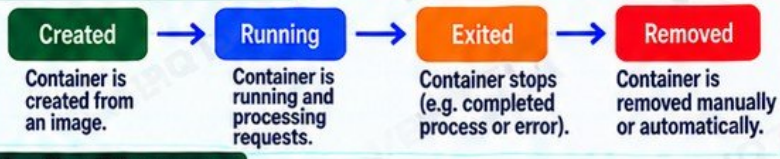
6. Troubleshooting Containers

Type of Issue	Command to Use	What It Does
Container exits immediately	`docker logs <container>`	Shows the reason for the exit.
Check container status	`docker ps -a`	Shows all containers (running and stopped).
Inspect container details	`docker inspect <container>`	Shows detailed configuration and logs.
Check port conflicts	`sudo lsof -i :8080`	Checks if the port is already in use.
Test connectivity inside container	`docker exec -it <container> bash`	Open a shell inside the container.
Network issues	`docker network ls` `docker network inspect <network>`	List networks and inspect network details.

7. Useful Commands Reference

Command	Description
`docker ps`	List running containers
`docker ps -a`	List all containers
`docker logs <container>`	Show container logs
`docker exec -it <container> bash`	Open a shell inside a container
`docker stop <container>`	Stop a container
`docker start <container>`	Start a container
`docker rm <container>`	Remove a container
`docker volume ls`	List volumes
`docker network ls`	List networks

8. Container Lifecycle



9. Key Takeaways

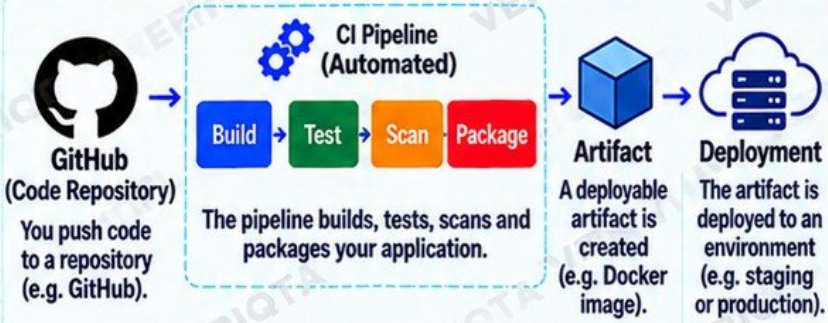
- Port publishing allows external access to containers.
- Volumes provide persistent storage.
- Container networks enable communication between containers.
- Logs and inspection tools help diagnose issues.
- Always check logs and container status when something goes wrong.
- You can run and troubleshoot a real containerized application using these skills.

CI/CD Fundamentals: From Code to Deployment

1. What Is CI/CD?

- Continuous Integration (CI) automatically builds, tests and checks code when changes are pushed.
- Continuous Delivery (CD) automatically packages and deploys the application to an environment after successful checks.
- It helps deliver high-quality software faster and more safely.
- It reduces manual work and human errors.
- It allows teams to ship changes frequently and with confidence.

2. CI/CD Pipeline Flow



3. Common Pipeline Stages

Build	Test	Scan	Package	Deploy
Compile code	Run automated tests (unit, integration)	Scan for security vulnerabilities	Create a deployable artifact (e.g. Docker image, JAR file)	Deploy to staging or production
Build a Docker image	Check code quality	Check dependencies for risks	Tag and store the artifact	Use deployment gates and approvals
Install dependencies	Fail if tests do not pass	Fail if critical issues are found		Verify the deployment

4. Example: CI/CD with GitHub Actions

```yaml
name: CI/CD Pipeline
on:
  push:
    branches: [ main ]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4
      - name: Build Docker image
        run: docker build -t myapp:latest .
      - name: Run tests
        run: docker run myapp:latest npm test
      - name: Push image to registry
        run: docker push myregistry/myapp:latest
```

5. Deployment Environments

Development	Staging	Production
Used by developers for active work.	A production-like environment.	Live environment used by real users.
Changes are deployed frequently.	Used for final testing before production.	Deployment should be safe, controlled and monitored.
May use automatic deployment.	Deployment may require approval.	Use approvals, tests and rollout strategies (e.g. blue/green, canary).

6. Why Failed Pipelines Should Block Changes

- Prevents broken code from reaching users.
- Stops security vulnerabilities from being deployed.
- Maintains application stability and reliability.
- Protects your users and your business.
- Enforces quality and consistency across the team.

7. Example: Build, Test and Deploy a Dockerized App

```bash
# 1. Build the image
docker build -t myapp:1.0 .

# 2. Run tests inside container
docker run --rm myapp:1.0 \
  npm test
```

```bash
# 3. Tag and push to registry
docker tag myapp:1.0 \
  myregistry/myapp:1.0
docker push myregistry/myapp:1.0
```

```bash
# 4. Deploy to server (example)
docker pull myregistry/myapp:1.0
docker stop myapp || true
docker rm myapp || true
docker run -d --name myapp -p 80:80 \
  myregistry/myapp:1.0
```

8. Key Tools Used in CI/CD

GitHub Actions · Jenkins · GitLab CI · Azure DevOps · Travis CI · CircleCI · Docker · Kubernetes

9. Key Takeaways

- CI/CD automates the entire delivery process.
- Common stages: build, test, scan, package, deploy.
- Automated tests and security checks are essential.
- Failed pipelines should block unsafe changes.
- A deployable artifact is created and deployed to an environment.
- CI/CD helps you deliver software faster, safer and more reliably.

Build Your First GitHub Actions Pipeline

1. What You Will Learn

- Understand workflows, jobs, steps and runners.
- Learn the purpose of .github/workflows/.
- Create a simple automated testing workflow.
- Understand triggers and workflow execution.
- Troubleshoot failed jobs using pipeline logs.
- Learn basic secret-handling practices.

2. How GitHub Actions Works



Code Push
You push code to a GitHub repository (e.g. main branch).



Triggers
A workflow is triggered by events (push, pull request, etc).



GitHub Actions
Runs your workflow on a runner. Executes jobs and steps.

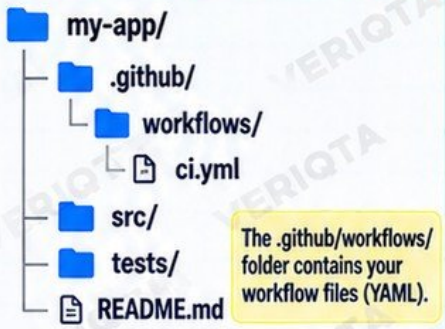


Tests and Build
Runs your tests, builds the application and performs the required tasks.



Result
Shows success or failure in the Actions tab.

3. Repository Structure



4. Example Workflow File (ci.yml)

```yaml
name: CI Pipeline

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
    - name: Checkout code
      uses: actions/checkout@v4

    - name: Set up Python
      uses: actions/setup-python@v5
      with:
        python-version: '3.11'

    - name: Install dependencies
      run: pip install -r requirements.txt

    - name: Run tests
      run: pytest
```

5. Workflow Triggers

Event	When It Runs
push	When you push code to a branch.
pull_request	When a pull request is opened, updated or reopened.
workflow_dispatch	Manual trigger (button).
schedule	Runs on a schedule (e.g. daily).

6. GitHub Actions Runners



GitHub-hosted runners
- Managed by GitHub
- Linux, Windows, macOS
- Free for public repos (limits apply)

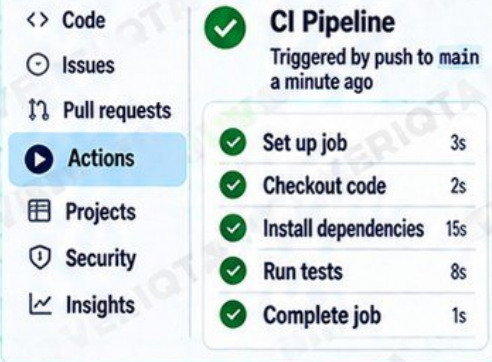


Self-hosted runners
- Run on your own servers
- More control
- Useful for private or special environments

7. Workflow Jobs and Steps



8. View Workflow Runs



9. Troubleshoot Failed Jobs



What to Do

1. Open the failed job and check the logs.
2. Read the error message carefully.
3. Verify your code, dependencies and environment.
4. Re-run the workflow after fixing the issue.

10. Secrets and Sensitive Data

- Store sensitive data in GitHub Secrets.
- Never hardcode passwords, API keys or tokens.
- Use secrets in your workflow with `${{ secrets.SECRET_NAME }}`.
- Limit access to only what is needed.
- Rotate secrets regularly.

```yaml
- name: Use a secret
  run: echo "API_KEY=${{ secrets.API_KEY }}"
```

11. Key Takeaways

- GitHub Actions helps you automate testing and deployment.
- Workflows are defined in `.yml` files inside `.github/workflows/`.
- You can run jobs on GitHub-hosted or self-hosted runners.
- Always check logs to troubleshoot failed jobs.
- Use secrets to keep sensitive information safe.
- Automated pipelines improve speed, quality and reliability.

Kubernetes Fundamentals for Beginners

1. What You Will Learn

- Understand container orchestration.
- Learn clusters, nodes, Pods, Deployments, and Services.
- Understand desired state and reconciliation.
- Learn why Kubernetes replaces failed Pods.
- See how the components work together.

2. What Is Kubernetes?

kubernetes

- Kubernetes (K8s) is an open-source container orchestration platform.
- It helps you deploy, manage, scale and run containerized applications.
- It keeps your application running even if containers or servers fail.
- It automates many operational tasks so you can focus on building and shipping software.

3. Key Components

Pod

- Smallest unit in Kubernetes.
- Runs one or more containers.
- Each Pod gets its own IP address.

Deployment

- Manages multiple replicas of Pods.
- Keeps the desired number of Pods running.
- Replaces failed Pods. automatically.

Service

- Provides a stable network endpoint for accessing Pods.
- Load balances traffic to healthy Pods.
- Can be exposed inside or outside the cluster.

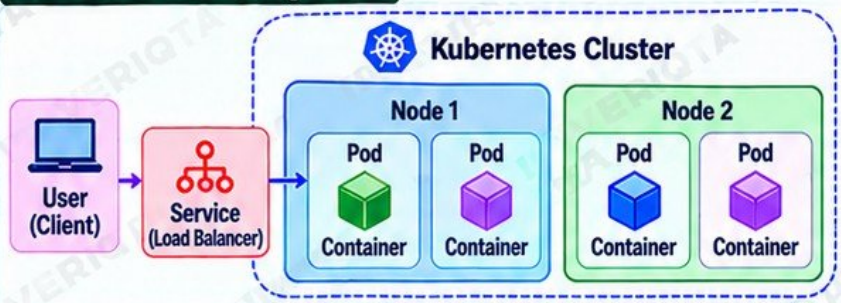
Node

- A worker machine (virtual or physical).
- Runs Pods.
- Managed by the control plane.

Cluster

- A set of nodes managed by Kubernetes.
- Runs all your applications.
- Includes a control plane and worker nodes.

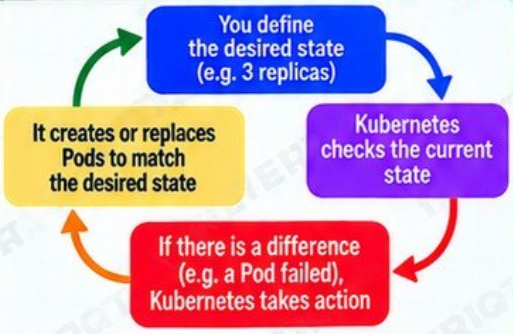
4. How It Works Together



5. Cluster → Node → Pod → Container

- **Cluster** — A Kubernetes cluster manages multiple nodes.
- **Node** — A node is a worker machine that runs Pods.
- **Pod** — A Pod is a group of one or more containers that run together.
- **Container** — A container is an isolated application with its own filesystem, process and runtime.

6. Desired State and Reconciliation



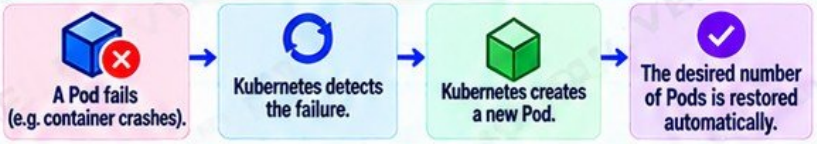
- You tell Kubernetes what you want (desired state).
- Kubernetes continuously checks the current state against the desired state.
- If a Pod fails, Kubernetes automatically creates a new one to maintain the desired number of replicas.
- This process is called reconciliation.

7. Example: Deployment YAML

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  replicas: 3 # desired number of Pods
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:1.25
        ports:
        - containerPort: 80
```

- Specifies the desired number of replicas.
- Labels to identify Pods.
- Container image to run.
- Exposes port 80 inside the container.

8. What Happens When a Pod Fails?



9. Key Takeaways

- Kubernetes runs and manages containerized applications.
- It uses the desired state and reconciliation loop.
- Deployments keep your application running.
- Services provide stable network access.
- Kubernetes automatically replaces failed Pods.
- It helps you build highly available and scalable applications.

Deploy and Troubleshoot Your First Kubernetes Application

1. What You Will Learn

- Understand Kubernetes manifests and YAML.
- Learn kubectl get, describe, apply, and logs.
- Deploy a simple application to a local cluster.
- Understand readiness and liveness probes.
- Troubleshoot ImagePullBackOff and CrashLoopBackOff.

2. Application Architecture



3. Example Kubernetes Deployment YAML

```yaml
apiVersion: apps/v1          # API version
kind: Deployment             # Resource type
metadata:
  name: myapp                # Deployment name
  labels:                    # Labels for the app
    app: myapp
spec:
  replicas: 2                # Number of Pods
  selector:
    matchLabels:             # Select Pods with this label
      app: myapp
  template:
    metadata:                # Pod template
      labels:
        app: myapp
    spec:
      containers:            # Container configuration
      - name: myapp          # Container name
        image: nginx:1.25    # Container image
        ports:
        - containerPort: 80  # Expose port 80
        readinessProbe:
          httpGet:           # Check if app is ready
            path: /
            port: 80
          initialDelaySeconds: 5
          periodSeconds: 10
        livenessProbe:
          httpGet:           # Check if app is alive
            path: /
            port: 80
          initialDelaySeconds: 15
          periodSeconds: 20
```

4. Deploy the Application

1. Save the YAML file (e.g. deployment.yaml).
2. Apply it to your cluster.
   ```
   kubectl apply -f deployment.yaml
   ```
3. Check that the resources are created.
   ```
   kubectl get all
   ```
4. Verify the Pods are running.
   ```
   kubectl get pods
   ```
5. Access the application (local cluster).
   ```
   kubectl port-forward svc/myapp 8080:80
   ```

Open your browser: http://localhost:8080

5. Essential kubectl Commands

Command	What It Does
kubectl get all	List all resources.
kubectl get pods	List Pods.
kubectl describe pod <pod>	Show detailed info about a Pod.
kubectl logs <pod>	View container logs.
kubectl apply -f <file.yaml>	Create or update resources.
kubectl delete -f <file.yaml>	Delete resources.
kubectl get svc	List Services.
kubectl get deployments	List Deployments.
kubectl rollout status deployment/<name>	Check rollout status.

6. Pod Status and Troubleshooting

Status	Meaning	What to Check
Running	Pod is running correctly.	No action needed.
Pending	Pod is waiting for resources.	Check node resources and events.
ImagePullBackOff	Cannot pull the container image.	Check image name, registry access, and image tag.
CrashLoopBackOff	Container keeps crashing.	Check container logs and application configuration.
Error	Pod failed to start.	Run kubectl describe pod <pod> and check events.

7. View Logs and Describe a Pod

```
# View container logs
kubectl logs myapp-7d5f6c8b7-abc123
/docker-entrypoint.sh: Starting nginx...
nginx: ready for connections
```

```
# Describe the pod for detailed information
kubectl describe pod myapp-7d5f6c8b7-abc123
Name:             myapp-7d5f6c8b7-abc123
Status:           Running
IP:               10.244.0.12
Containers:
  myapp:
    Image:        nginx:1.25
    State:        Running
    Ready:        True
Events:
  Normal  Pulled  Successfully pulled image "nginx:1.25"
```

8. Readiness vs Liveness Probes

Readiness Probe

- Checks if the application is ready to receive traffic.
- If it fails, the Pod is removed from the Service endpoints.
- Helps prevent sending traffic to an unhealthy Pod.

Liveness Probe

- Checks if the application is still running.
- If it fails, Kubernetes restarts the container.
- Helps recover from application failures automatically.

9. Common Issues and Fixes

Issue	Cause	How to Fix
ImagePullBackOff	Wrong image name or no registry access.	Check the image name, tag and registry credentials.
CrashLoopBackOff	Application crash or bad configuration.	Check logs, fix configuration and redeploy.
Pending	Not enough resources.	Check node resources or reduce resource requests.

10. Key Takeaways

- Kubernetes manifests define how your application runs.
- kubectl helps you manage and troubleshoot resources.
- Readiness and liveness probes improve application reliability.
- ImagePullBackOff and CrashLoopBackOff are common and can be diagnosed using logs and events.
- Kubernetes automatically replaces failed Pods to maintain the desired state.

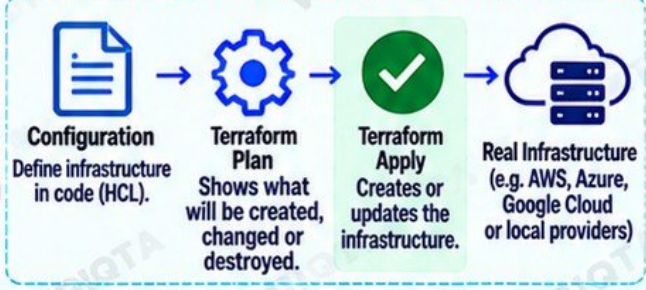
Terraform and Infrastructure as Code

1. What You Will Learn

- Understand Infrastructure as Code (IaC).
- Learn providers, resources, variables, and state.
- Understand terraform init, plan, and apply.
- Learn why infrastructure changes require review.
- Practice creating a small local Terraform configuration.
- Understand state protection and safe cleanup.

2. What Is Terraform?





3. Key Concepts

Providers	Resources	Variables	State
Plugins that allow Terraform to interact with cloud platforms (e.g. AWS, Azure, Google Cloud) or other services.	Define the infrastructure objects to create (e.g. VM, network, storage).	Allow you to parameterize values (e.g. region, instance type).	Keeps track of the real infrastructure. Used to detect changes. Must be protected (e.g. remote state with locking).

4. Example: Terraform Configuration (Local)

```hcl
# main.tf
terraform {
  required_providers {
    docker = {
      source = "kreuzwerker/docker"
      version = "~> 3.0"
    }
  }
}
provider "docker" {}

resource "docker_image" "nginx" {
  name = "nginx:latest"
}

resource "docker_container" "web" {
  name  = "nginx-web"
  image = docker_image.nginx.name
  ports {
    internal = 80
    external = 8080
  }
}
```

Define required provider and version.

Configure the provider (in this example, Docker for local testing).

Resource to pull the Nginx image.

Resource to create a container and publish port 8080.

5. Terraform Workflow



6. Example: Commands and Output

```
# 1. Initialize
terraform init
Initializing the backend...
Initializing provider plugins...
Terraform has been successfully initialized!

# 2. Plan
terraform plan
Plan: 2 to add, 0 to change, 0 to destroy.

# 3. Apply
terraform apply
Do you want to perform these actions?  Enter yes
Apply complete! Resources: 2 added, 0 changed, 0 destroyed.

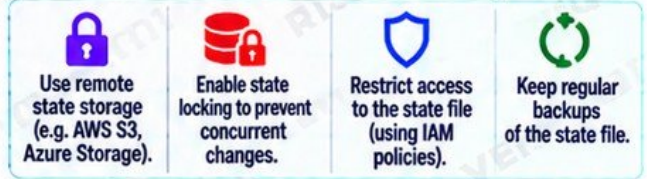
# 4. Check resources
docker ps
CONTAINER ID   IMAGE          PORTS                    NAMES
a1b2c3d4e5f6   nginx:latest   0.0.0.0:8080->80/tcp     nginx-web

# 5. Cleanup
terraform destroy
Destroy complete! Resources: 2 destroyed.
```

7. Why Infrastructure Changes Require Review

- Prevents accidental deletion of critical resources.
- Helps catch security, cost, and compliance issues.
- Allows team review before changes are applied.
- Reduces risk of misconfiguration and downtime.
- Provides an audit trail through version control.

8. State Protection Best Practices



9. Key Takeaways

- Terraform lets you manage infrastructure using code (IaC).
- You define providers, resources, variables, and state in configuration files.
- Use terraform init, plan, and apply to create and manage infrastructure.
- Always review changes before applying them.
- Protect your state file and use remote state with locking.
- Clean up resources when they are no longer needed to avoid unnecessary costs.

Monitoring, Logging, and Production Troubleshooting

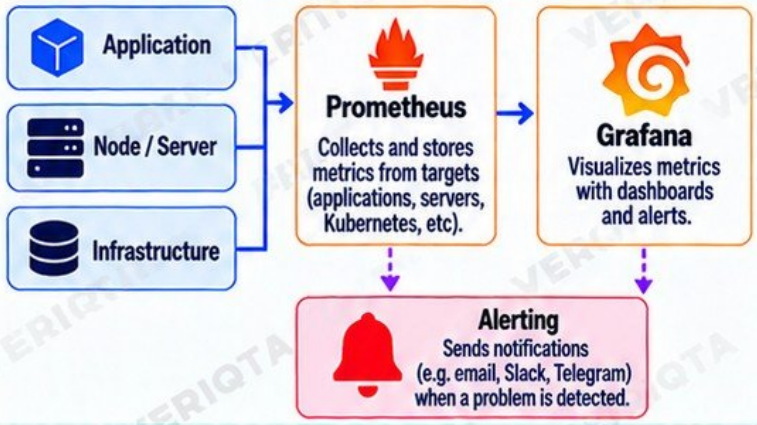
1. What You Will Learn

- Understand metrics, logs, and traces.
- Learn the purpose of Prometheus and Grafana.
- Understand CPU, memory, latency, traffic, and error rates.
- Learn basic alerting and incident response.
- Practice investigating a simulated application failure.
- Understand why engineers verify recovery.

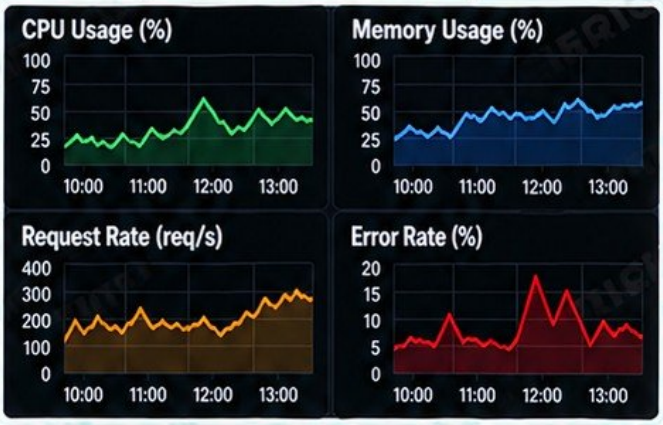
2. Observability Overview

Metrics	Logs	Traces
Numeric data over time (e.g. CPU, memory, latency, request rate).	Text records of events (e.g. application logs, system logs).	Follow a request across multiple services.
Used to monitor health and performance.	Used to understand what happened.	Show the full journey and where time is spent.
Collected by Prometheus and visualized in Grafana.	Help with debugging and incident investigation.	Help identify latency and failures in distributed systems.

3. Monitoring Stack Architecture



4. Example: Grafana Dashboard



5. Useful Prometheus Metrics (Examples)

Metric	Meaning	Example Use
node_cpu_usage_seconds_total	CPU usage	Monitor server CPU.
node_memory_MemAvailable_bytes	Available memory	Detect memory pressure.
http_requests_total	Number of requests	Monitor traffic.
http_request_duration_seconds	Request latency	Identify slow requests.
http_requests_total{status=~"5..."}	Error rate (5xx)	Monitor application errors.

6. Example Logs (Application)

```
2026-10-15 12:01:22  INFO   Starting application...
2026-10-15 12:01:25  INFO   Connected to database
2026-10-15 12:05:13  ERROR  Failed to fetch data:
                            timeout after 5s
2026-10-15 12:05:14  WARN   Retrying request (1/3)
2026-10-15 12:05:19  ERROR  Database connection failed
2026-10-15 12:05:20  INFO   Recovered and reconnected
```

7. Investigate a Simulated Failure

1. Check the alerts in Grafana.
2. Look at metrics (CPU, memory, request rate, error rate).
3. Check application logs for errors.
4. Identify the root cause (e.g. high CPU, crashed container, database failure).
5. Fix the issue (e.g. restart service, update configuration).
6. Verify recovery (metrics return to normal, no more errors).

8. Common Issues and What to Check

Issue	Possible Cause	What to Check
High CPU usage	Infinite loop, high traffic	CPU metrics, logs, recent changes
High memory usage	Memory leak	Memory metrics, logs, application code
High latency	Slow database, external API	Latency metrics, traces, logs
High error rate (5xx)	Application error, dependency failure	Error metrics, logs, service health
No data in Grafana	Prometheus not scraping	Prometheus targets, configuration
Container keeps restarting	Application crash	Pod logs, events (kubectl describe)

9. Incident Response Flow



10. Key Takeaways

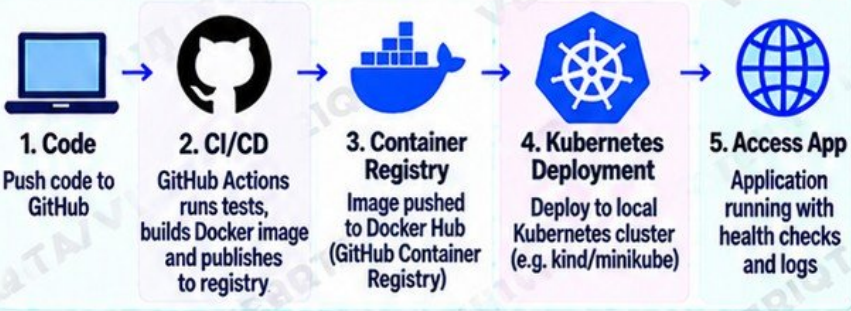
- Metrics, logs, and traces give you visibility.
- Prometheus collects metrics and Grafana visualizes them.
- Monitor CPU, memory, latency, traffic, and error rates.
- Set up alerts to detect problems early.
- Follow a clear incident response process.
- Always verify recovery before closing the incident.

Your First End-to-End DevOps Portfolio Project

1. What You Will Build

- GitHub repository and application source code.
- Docker containerization.
- GitHub Actions automated testing.
- Container image publishing.
- Deployment to a local Kubernetes cluster.
- Application health checks and logs.
- README documentation and architecture diagram.

2. Project Architecture Overview



3. Project Application

Example: Simple Web Application (Python Flask)

- A simple web app with a home page and /health endpoint.
- Easy to containerize and deploy.
- Used to demonstrate the full DevOps workflow.

4. Repository Structure

```
my-devops-project/
├── app/
│   ├── app.py
│   └── requirements.txt
├── .github/workflows/
│   └── ci.yml
├── k8s/
│   ├── deployment.yaml
│   └── service.yaml
├── Dockerfile
└── README.md
```

5. Dockerfile (Application Image)

```dockerfile
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app/ .
EXPOSE 5000
CMD ["python", "app.py"]
```

6. GitHub Actions Workflow (CI/CD)

```yaml
name: Build and Deploy

on:
  push:
    branches: [ main ]

jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.11'
      - name: Install dependencies
        run: pip install -r requirements.txt
      - name: Run tests
        run:  pytest
      - name: Build Docker image
        run: docker build -t ${{ github.repository }}:latest .
      - name: Push to registry
        run: docker push ${{ github.repository }}:latest
```

7. Kubernetes Manifests

deployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp
spec:
  replicas: 2
  selector:
    matchLabels:
      app: myapp
  template:
    metadata:
      labels:
        app: myapp
    spec:
      containers:
      - name: myapp
        image: <your-image>:latest
        ports:
        - containerPort: 5000
        readinessProbe:
          httpGet:
            path: /health
            port: 5000
```

service.yaml

```yaml
apiVersion: v1
kind: Service
metadata:
  name: myapp-service
spec:
  type: NodePort
  selector:
    app: myapp
  ports:
  - port: 80
    targetPort: 5000
    nodePort: 30080
```

Deploy to Local Cluster

```
kubectl apply -f k8s/
kubectl get pods
kubectl get svc
kubectl logs -f deployment/myapp
```

8. Access and Verify the Application

```
# Get the service URL (for minikube)
minikube service myapp-service

# Or access using NodePort
http://localhost:30080
```



9. View Logs and Health Checks

```
# View application logs
kubectl logs -f deployment/myapp
# Check health endpoint
curl http://localhost:30080/health
{
  "status": "healthy",
  "message": "Application is running"
}
```

10. Key Takeaways

- You built a complete end-to-end DevOps project.
- You used GitHub, Docker, GitHub Actions and Kubernetes.
- You deployed a real application.
- You verified application health and logs.
- You documented the project.
- This project can be added to your portfolio to demonstrate real-world DevOps skills.

Build a Professional GitHub Portfolio and Resume

1. What You Will Learn

- Organize GitHub repositories professionally.
- Write useful READMEs with setup and verification instructions.
- Document architecture, technical decisions, and troubleshooting.
- Present completed projects as evidence of practical skills.
- Write a junior DevOps resume with truthful, measurable achievements.
- Understand what employers look for in entry-level candidates.

2. Professional GitHub Profile Setup

your-username
DevOps Learner | Building in Public

Learning and building real-world DevOps projects. Sharing my journey, documentation and practical examples.

Lagos, Nigeria
https://github.com/your-username

Edit profile

Pinned Repositories

devops-project-1 Public
End-to-end DevOps project with Docker, CI/CD and Kubernetes.
Docker Kubernetes GitHub Actions
★ 120 35

infrastructure-labs Public
Terraform and IaC examples.
Terraform AWS DevOps
★ 85 28

3. Repository Structure (Example)

```
my-devops-project/
├── app/                    → Application source code (e.g. Python, Node.js, etc.)
│   ├── src/
│   └── Dockerfile          → Docker configuration
├── .github/
│   └── workflows/          → GitHub Actions workflows (CI/CD)
│       └── ci.yml
├── k8s/                    → Kubernetes manifests
│   ├── deployment.yaml
│   └── service.yaml
├── docs/                   → Documentation (architecture, decisions, troubleshooting)
│   ├── architecture.md
│   └── troubleshooting.md
├── README.md               → Main project documentation with setup and verification steps
└── requirements.txt        → Project dependencies
```

4. A Good README Example

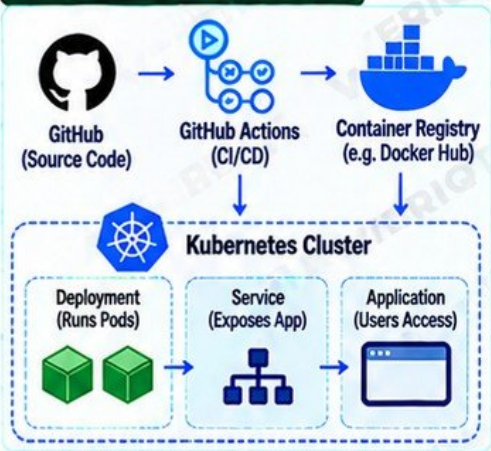
```
# DevOps Project

Python | Docker | Kubernetes | CI/CD | License MIT

A simple web application deployed using Docker, GitHub Actions and Kubernetes.

## 1. Project Overview
## 2. Architecture Diagram
## 3. Tech Stack
## 4. Prerequisites
## 5. Setup Instructions
## 6. Run Locally (Docker)
## 7. Deploy to Kubernetes
## 8. Verification Steps
## 9. Troubleshooting
## 10. Key Learnings
```

5. Architecture Diagram (Include in README)



6. Example README Sections

- Project description and purpose.
- Architecture diagram.
- Tech stack and tools used.
- Step-by-step setup instructions.
- How to run and test the application.
- How to deploy (Docker and Kubernetes).
- Verification steps (with expected output).
- Troubleshooting common issues.
- Screenshots and demo (optional).
- Key learnings and next steps.

7. Writing a Junior DevOps Resume

Your Name
Junior DevOps Engineer

Lagos, Nigeria | your.email@gmail.com
linkedin.com/in/yourname | github.com/your-username

SUMMARY
Motivated and detail-oriented DevOps learner with hands-on experience in Linux, Docker, Kubernetes, GitHub Actions and Terraform. Built and deployed real-world projects demonstrating CI/CD, automation. and infrastructure as code.

TECHNICAL SKILLS
Linux | Git | Docker | Kubernetes | GitHub Actions | Terraform
Prometheus | Grafana | Python | YAML

PROJECTS
My DevOps Project (GitHub link)
- Built a containerized web application using Docker.
- Implemented CI/CD with GitHub Actions.
- Deployed to a local Kubernetes cluster (Minikube).
- Added monitoring with Prometheus and Grafana.

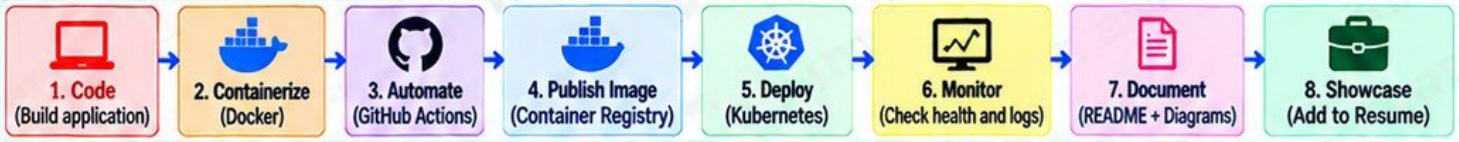
8. Resume Best Practices

- Keep it clear, concise and one to two pages.
- Highlight practical projects with measurable outcomes.
- Include relevant technical skills.
- Use action verbs (built, deployed, automated, configured, monitored, etc.).
- Provide links to your GitHub repositories.
- Be honest and only include what you know.
- Tailor your resume to the job description.

9. What Employers Look For (Entry-Level)

- Practical hands-on projects.
- Understanding of core DevOps tools.
- Ability to work with Linux and cloud basics.
- Good documentation and communication.
- Problem-solving and troubleshooting skills.
- Willingness to learn and improve.

10. Complete Portfolio Flow



Your First DevOps Job: Interview and Application Roadmap

1. What You Will Learn

- Identify suitable junior DevOps, cloud support, infrastructure, and operations roles.
- Understand common Linux, networking, Git, Docker, CI/CD, and cloud interview topics.
- Prepare to explain your projects and troubleshooting decisions.
- Practice technical interview questions and behavioral scenarios.
- Build a realistic application and interview preparation routine.
- Understand that practical evidence matters more than memorizing tool names.

2. Suitable Entry-Level Roles

Junior DevOps Engineer	Cloud Support Engineer	Infrastructure Engineer (Junior)	Site Reliability Engineer (Associate)	IT Operations Engineer
Work on CI/CD, infrastructure, automation and monitoring.	Support cloud platforms (e.g. AWS, Azure, GCP).	Manage and maintain servers, networks and infrastructure.	Focus on reliability, monitoring and incident response.	Keep systems running, handle incidents and support applications.

3. Key Interview Topic Areas

Linux
- File system
- users & permissions
- processes
- systemd
- troubleshooting

Networking
- TCP/IP
- ports and protocols
- DNS
- firewalls
- troubleshooting

Git
- basic commands
- branching
- merging
- pull requests
- trollaboration

Docker
- images
- containers
- volumes
- networks
- troubleshooting

Kubernetes
- pods
- services
- deployments
- configmaps
- troubleshooting

CI/CD
- GitHub Actions
- pipelines
- build, test, deploy
- artifacts
- troubleshooting

Cloud (AWS/Azure/GCP)
- core services
- compute
- storage
- networking
- identity and access

Monitoring & Logging
- Prometheus
- Grafana
- logs
- metrics
- alerting

Infrastructure as Code
- Terraform basics
- providers
- resources
- state
- plan and apply

Security Basics
- IAM
- secrets
- image security
- basic best practices

4. Explaining Your Projects

1. What problem were you solving?
2. What tools and technologies did you use?
3. What was your architecture?
4. What were the key decisions and trade-offs?
5. What challenges did you face?
6. How did you troubleshoot and fix issues?
7. What did you learn?
8. How could you improve it?

5. Technical Interview Practice

- Answer common technical questions.
- Do hands-on labs and real scenarios.
- Explain your thought process clearly.
- Practice troubleshooting out loud.

6. Behavioral Interview Preparation

- Tell me about yourself.
- Why do you want a DevOps role?
- Describe a challenge you faced and how you solved it.
- How do you handle incidents and pressure?
- How do you work in a team?
- Where do you see yourself in 2–3 years?

7. Application Roadmap

1. Build and complete 2–3 strong projects.
2. Create a professional GitHub profile (with good READMEs).
3. Write a focused, truthful resume.
4. Apply to relevant entry-level roles.
5. Prepare for interviews (technical + behavioral).
6. Continue learning and improving.

8. Sample Weekly Preparation Plan

Day	Focus
Mon	Linux and command line practice
Tue	Networking and cloud basics
Wed	Docker and Kubernetes labs
Thu	CI/CD and Terraform practice
Fri	Monitoring, logging and troubleshooting
Sat	Technical interview questions
Sun	Behavioral practice and resume/job applications

9. Entry-Level Resume Structure (Example)

YOUR NAME
Junior DevOps Engineer

youremail@gmail.com
Lagos, Nigeria
linkedin.com/in/yourname
github.com/your-username

Summary	Technical Skills	Projects	Experience / Education
Motivated DevOps learner with hands-on experience in Linux, Docker, Kubernetes, GitHub Actions and cloud platforms.	Linux | Git | Docker Kubernetes | CI/CD Terraform AWS (basic) Monitoring | Scripting	My DevOps Project - Built a containerized web application. - Implemented CI/CD with GitHub Actions. - Deployed to Kubernetes. - Added monitoring with Prometheus and Grafana.	N/A or relevant coursework, certifications or internships.

10. Key Takeaways

- Practical projects and evidence matter most.
- Be able to explain your decisions and troubleshooting.
- Focus on problem-solving, not just tool names.
- Build a professional GitHub portfolio and resume.
- Practice both technical and behavioral interviews.
- Apply consistently and keep improving.